IIIS-NLP Assignment 2

Fall 2026

Assignment Guidelines

- The assignment is due on **10 p.m. Beijing Time on December 15**. We have a strict late policy: penalties increase to 10%, 20%, 50%, 100% for each day, where the number of days are rounded up.
- The assignment is a **coding assignment**. Submit a zip file containing your code/output and a report. If you choose Part 3, you will need to submit your **wandb log**.
- The full score is **100 points**. Partial credit will be given for partially correct answers. Double-check your submission to prevent typos and compilation errors.
- The provided AutoDL resource, discussion between classmates, online search, and the use of free AI or free online assets are allowed. **Other than AutoDL, non-free AI or assets are not allowed**. At the end of the report, you should acknowledge all collaborators, online sources, and state how and how much you have used AI.
- You must choose only one problem from Part3, or Part4. If you submit more than one problem, only one will be scored. Also note that the homework and the project will be graded seperately. i.e., There will be no penalty for overlap between homework and project.

Part 1 Pretrain Transformer (30 points)

You'll train a Transformer to perform a task that requires accessing world knowledge—knowledge not provided in the task's training data (especially if generalization beyond the training set is needed). Initially, you'll observe that the Transformer largely fails at the task. Then, you'll learn to pretrain the Transformer on Wikipedia text, which contains relevant world knowledge. Fine-tuning the Transformer on this knowledge-intensive task afterward allows it to leverage the knowledge gained during pretraining. This approach improves the performance significantly.

(a) **Review NameDataset:** Our goal is to predict birthplace based on name, using the `NameDataset` class in `dataset.py`. Run `python dataset.py namedata` to inspect sample data.

(b) **Construct your model:** Construct a decoder-only model. You may want to complete several coding blocks in `models.py` and `attention.py`. Note that you can try different model architectures, which may help you in the later steps. When implementing the forward pass, use the NLL loss.

(c) **Implement Finetuning (without pretraining) (10 points):** Modify `run.py` for finetuning a Transformer on `NameDataset`. After training, you should report your prediction score on the dev set. You can refer to `run_finetune.sh` and `run_evaluate.sh` for launching details.

(d) **Pretrain:** Implement `getitem()` in `CharCorruptionDataset` for a span corruption task, as per instructions in `dataset.py`. CharCorruptionDataset is a class that generates examples for a simplified span corruption objective used in language model training, and it is explained in detail in the Python document. Debug with the command `python dataset.py charcorruption`. Then you should run `run_pretrain.sh` to get a pretrained model. Your pretrain should finish in less than an hour (on cpu).

(e) **Finetune at pretrained model, and Make predictions (20 points):** Finetune at the pretrained model, and also report your prediction accuracy score on the dev set. We expect the final performance will exceed 12%. If you encounter difficulties, consider experimenting with alternative architectures and hyperparameters.

In addition to the report, you should also submit the following modified code:

```
part1/
|-- models.py
|-- attention.py
|-- run.py
‘-- dataset.py
```

Part 2: A Mini Web-Data Filtering Pipeline (30 points)

Large language models are trained on web-scale corpora, but raw web pages cannot be used directly. Before pretraining, a data pipeline normally performs text extraction, privacy protection, safety filtering, quality filtering, and deduplication. In this task, you will implement a small, deterministic version of such a pipeline. This part is intended to help you understand how training data are cleaned for modern large language models.

The corpus contains 100 locally bundled HTML pages, with IDs `web_001` through `web_100`. They are real HTTP response bodies in one fixed `CC-MAIN-2026-39` WARC file. Every HTML file is byte-for-byte identical to its frozen response body. The pages are randomly selected and vary in topics and length.

The complete task runs on CPU and is designed to take approximately two hours.

Required Processing Order

Apply the stages in exactly this order:

`HTML extraction` → `PII masking` → `offensive-language filtering` → `quality filtering` → `near-duplicate removal` → `JSONL output`

This order is part of our specification. A document could be modified or removed. Personally Identifiable Information (PII) masking happens before the safety and quality decisions, and a removed document does not enter any later stage. Deduplication receives only documents that passed both filters. The final output contains extracted, PII-masked text—not the normalized representation used for deduplication. A document removed at one stage does not reach any later stage.

We regard your implementation as correct if each part of it has passed the corresponding unit test.

Starter Repository and Data

The starter repository contains:

```
starter/
|-- mini_data_pipeline/
| |-- __init__.py
| |-- __main__.py
| |-- pipeline.py # complete the TODO blocks
| `-- provided.py # provided; do not modify
|-- data/
| |-- manifest.jsonl
| |-- SOURCES.json
| `-- pages/
|-- tests/
|-- pyproject.toml
`-- requirements.txt
```

Each line of `manifest.jsonl` has four fields:

```
{"id":"web_001","path":"pages/web_001.html",
 "encoding":"utf-8","sha256":"..."}
```

The provided loader validates IDs, relative paths, encodings, and SHA256 checksums. Document IDs are unique, case-sensitive ASCII strings and are sorted using Python's default lexicographic order. Decode each file using the encoding specified in the manifest.

From the `starter/` directory, create and activate a dedicated Conda environment, then install the pinned dependencies:

```
conda create -n hw2-data-pipeline python=3.11 pip -y
conda activate hw2-data-pipeline
python -m pip install -r requirements.txt
```

Read the instructions below, and implement the functions marked `TODO` in `pipeline.py`. You may add imports and private helper functions to that file, but do not change the supplied public function signatures or constants, and do not modify the data, provided helper, or test files. After implementing the functions, run:

```
python -m pytest -q
```

to check your implementation.

HTML to Plain Text (10%)

Implement:

```
def extract_text(html_bytes: bytes, *, encoding: str) -> str:
    """Extract and canonicalize visible text from one HTML byte string."""
```

Decode the bytes using the supplied encoding. Import `extract_plain_text` as follows:

```
from resiliparse.extract.html2text import extract_plain_text
```

Call it with exactly these options[1]:

```
preserve_formatting=True
main_content=False
list_bullets=False
alt_texts=False
links=False
form_fields=False
noscript=False
comments=False
post_meta=False
hidden_elements=False
```

Then canonicalize the extracted string with the following steps:

1. Convert CRLF and CR line endings to LF.
2. Split only on LF. Within each line, replace every consecutive horizontal whitespace[2] defined above with one ASCII space.
3. Strip whitespace from each line.
4. Remove empty lines.

[1] https://resiliparse.chatnoir.eu/en/stable/man/extract/html2text.html
[2] `[^\S\r\n]+`

5. Join the remaining lines using one LF character.

Return a string with no leading or trailing whitespace. If the page contains no visible text, return the empty string.

Personally Identifiable Information (PII) Masking (10%)

Implement:

```
def mask_pii(text: str) -> str:
    """Replace supported PII with fixed placeholders."""
```

Email address, telephone number and the IPv4 address are personally identifiable information, and you should mask them with placeholders:

```
|||EMAIL_ADDRESS|||
|||PHONE_NUMBER|||
|||IP_ADDRESS|||
```

Return only the masked string.

Grammar of email address, telephone number and the IPv4 address are listed below. Its recommended to use regular expression. Matches should be non-overlapping and left-to-right.

Email grammar

Use the following pseudo-regular grammar:

```
LETTER := "A" | ... | "Z" | "a" | ... | "z"
DIGIT := "0" | ... | "9"
LOCAL_CHAR := LETTER | DIGIT | "." | "_" | "%" | "+" | "-"
DOMAIN_CHAR := LETTER | DIGIT | "." | "-"
EMAIL := LOCAL_CHAR+ "@" DOMAIN_CHAR+ "." LETTER LETTER+
```

Here, `|` denotes alternation and `+` means one or more repetitions. This intentionally permissive detector is simplified from RFC-5322[3].

Telephone grammar

Support these layouts, where `d` denotes one ASCII digit from `0` through `9`:

```
ddd-ddd-dddd
ddd ddd dddd
(ddd) ddd-dddd
+1 ddd-ddd-dddd
+1 (ddd) ddd-dddd
```

[3]https://www.rfc-editor.org/info/rfc5322/

A match must not be immediately preceded or followed by an ASCII digit. Other formats are not required.

IPv4 grammar

An IPv4 address consists of four decimal components joined by periods. Each component is an integer from 0 through 255, written using ASCII digits. Leading zeros are not allowed unless the component is exactly `0`.

Formally, use the following pseudo-regular grammar:

```
DIGIT := "0" | ... | "9"
NONZERO := "1" | ... | "9"
OCTET := "0" | NONZERO | NONZERO DIGIT
         | "1" DIGIT DIGIT
         | "2" ("0" | ... | "4") DIGIT
         | "25" ("0" | ... | "5")
IPV4 := OCTET "." OCTET "." OCTET "." OCTET
```

Shown above is a way to implement this grammar directly. Alternatively, you may use a more vague regular expression first to find plausible candidates with format `num.num.num.num` and then check[4] each number is less or equal than 255.

Offensive-Language Detection (20%)

This stage uses the fixed, CPU-only `alt-profanity-check` package[5]. It supplies a lightweight bag-of-words linear classifier. Import:

```
from profanity_check import predict_prob
```

Then implement the existing function:

```
def is_harmful(text: str) -> bool:
    ...
```

Call `predict_prob([text])`, and return True if the returned probability is ≥ 0.5.

Quality Filtering (20%)

Implement:

```
def passes_quality_filter(text: str) -> bool:
    ...
```

Normalize the document to Unicode NFC before measuring it. A document passes only if all four rules hold:

[4] https://docs.python.org/3/library/re.html#re.sub
[5] https://pypi.org/project/alt-profanity-check/

1. token count is between 50 and 10,000, inclusive.
2. mean token length is between 3 and 10 Unicode code points, inclusive.
3. at most 30% of non-empty lines end with `...` or `U+2026 HORIZONTAL ELLIPSIS`.
4. at least 80% of tokens contain a Unicode alphabetic character according to `str.isalpha()`.

Use `text.split()` to split the document into tokens and `text.splitlines()` to split the document into lines. Each line should be striped. An empty document should not pass.

Near-Duplicate Detection (20%)

Normalization

At the beginning of duplicate detection, you should normalize text. Note that this normalization is only for dedup and in this step we do not change the text itself.

Implement `normalize_for_dedup(text)` using exactly this order:

1. Unicode NFD normalization[6];
2. `str.lower()` (not `casefold`);
3. delete all diacritical marks, i.e., every character whose Unicode category is `Mn`;
4. delete all punctuation, i.e., every character whose Unicode category begins with `P`;
5. replace each consecutive sequence of Unicode whitespace characters (`"\s+"`) with one ASCII space;
6. strip leading and trailing whitespace.

Word n-grams

Implement:

```
def word_ngrams(text: str, n: int = 5) -> set[tuple[str, ...]]:
    ...
```

Normalize the text with `normalize_for_dedup`, split it on ASCII spaces, calculate and return the set of contiguous word n-grams. In the output, repeated n-grams should occur only once. Raise `ValueError` if `n` is not a positive integer. Return an empty set if the normalized text has fewer than n words.

For example, recall that the word 3-grams of `a language model learns from text` are:

```
{
    ("a", "language", "model"),
    ("language", "model", "learns"),
    ("model", "learns", "from"),
    ("learns", "from", "text"),
```

[6]Normalization Form D (NFD) canonically decomposes a precomposed character into a base character followed by its combining mark(s).

```
}
```

Jaccard similarity

Implement:

```
def jaccard_similarity(a: set, b: set) -> float:
    ...
```

For two non-empty sets A and B,

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

If both sets are empty, return 1.0. If exactly one set is empty, return 0.0.

Deduplication

Implement:

```
def deduplicate_documents(
    documents: list[Document],
    n: int = 5,
    threshold: float = 0.8,
) -> tuple[list[Document], list[str]]:
    ...
```

Raise `ValueError` if `n` is not a positive integer. The threshold must be a finite `float` lying in $[0, 1]$, otherwise raise `ValueError`. Process documents in ascending ID order and maintain a list of retained representative documents:

1. Compare the current document with every representative document retained so far.
2. Keep it if every Jaccard similarity between the word n-grams of the current document and the word n-grams of the retained document is $J < 0.8$.

Return the retained documents and a list containing the ID of every **removed** document. For example, the second returned value may be:

```
["web_002", "web_005"]
```

Both returned lists must be in ascending document-ID order.

End-to-End Pipeline (20%)

Implement:

```
def run_pipeline(
    input_dir: str,
    output_path: str,
) -> None:
```

```
    ...
```

You may use the provided manifest loader and canonical writer. Write retained documents to a UTF-8 JSONL, sorted by ID:

```
{"id":"web_001","text":"..."}
{"id":"web_007","text":"..."}
```

The provided writer fixes key order, compact separators, LF line endings, and one final newline after every record.

Load the manifest, which is already returned in ascending ID order. For each document, extract text and mask PII. Pass the masked text to the offensive-language detection, then apply the quality filter only to accepted documents. Run `deduplicate_documents` with its default arguments on the remaining documents. Output only retained documents, using their extracted and PII-masked text.

Run the full pipeline with:

```
python -m mini_data_pipeline \
  --input-dir data \
  --output-path output/cleaned.jsonl
```

Submission

Add the following files to your HW2 submission:

```
part2/
|-- mini_data_pipeline/
| `-- pipeline.py
`-- output/
    `-- cleaned.jsonl
```

Do not submit any other file, and no need to write anything in the report.

Grading

For each component, you will receive the corresponding credit if your implementation faithfully follows the requirements in this handout and passes its unit test.

Repeat: You must choose only one problem from Part3 or Part4. If you submit more than one problem, only one will be scored. Also note that the homework and the project will be graded seperately. i.e., There will be no penalty for overlap between homework and project.

Part 3: How to train a large language model (40 points)

Like in all areas of Artificial Intelligence, when training large language models we often make use of open-source repositories to avoid rewriting common components such as SFT and RL implementations. This part of the assignment aims to develop your ability to utilize and modify open-source frameworks, including environment configuration.

Please note that, in general, library versions often conflict between different repos, so it's recommended to create a separate conda environment for each repo. In addition, some libraries may contain unresolved bugs. If you run into issues, you can first check the repo's issues and PRs to see whether anyone has already mentioned them. Also, when using services such as wandb, Hugging Face, or GitHub, you may encounter network issues. There are several ways to work around them. For example, using wandb's offline mode and uploading later, or using Hugging Face's mirror sites, and so on. If you still can't resolve the problem, please contact the TA in the WeChat group.

In **Part 3-1**, you are required to set up and configure `LLaMA-Factory` (https://github.com/hiyouga/LLaMA-Factory) and `Lighteval` (https://github.com/huggingface/lighteval). In **Part 3-2**, you will need to configure `verl` (https://github.com/volcengine/verl).

For this part, you are not required to submit any code. However, you must provide your training and evaluation `wandb` logs in the form of a `wandb.ai` link.

We recommend using the **Ubuntu** operating system and a GPU with **CUDA** support for this part. Training on other operating systems and GPU architectures is also feasible, as long as the environment is properly configured.

Special Notice for Students Using the Provided AutoDL Computing Resources: Due to the constrained space of the system disk (30G), it is highly recommended to relocate packages and environments to the data disk (default 50G, expandable up to 2T which should be sufficient). Specifically, add the following configuration to your .condarc file:

envs_dirs:
- /root/autodl-tmp/miniconda3
pkgs_dirs:
- /root/autodl-tmp/miniconda3

Additionally, remember to regularly monitor disk usage with the command:

du -sh /path/to/your/directory

Before renting GPUs on AutoDL, be sure to carefully review the guidelines in the repositories to avoid potential CUDA memory issues.

Part 3-0: Use `wandb.ai` for Experiment Logging

`wandb.ai` is one of the most widely used platforms for recording experiment logs. To use `wandb`, you first need to register an account at https://wandb.ai, and then obtain

an API key from https://wandb.ai/authorize. Before starting each experiment, make sure to set the environment variable `WANDB_API_KEY` to the API key you obtained, and set `WANDB_PROJECT` to `NLP2026_<your_student_ID>`.

All three repositories we will use already support automatic logging and uploading of training logs to `wandb`. Each repository provides detailed documentation explaining how to upload training logs to `wandb`, and we will also provide a brief explanation later.

Now, please create a `wandb` project on https://wandb.ai named `NLP2026_<your_student_ID>` and set its visibility to **public** or **open**. For example: https://wandb.ai/xkev/NLP2026_2023011416.

All of your subsequent experiments must be recorded. Provide the link to this project in the report.

Part 3-1: SFT (30 points)

In this section, you will improve the performance of the `Qwen2.5-0.5B-Instruct` model (https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct) on the `GSM-8K` dataset through supervised fine-tuning (SFT).

(a) **Evaluate the base model on the GSM-8K dataset (5 point):** Use the `lighteval` framework to evaluate the performance of `Qwen2.5-0.5B-Instruct` on the `GSM-8K` dataset. After correctly installing the `lighteval` framework, you can test the model using the following script:

```
WANDB_API_KEY=YOUR_API_KEY \
WANDB_PROJECT=NLP2026_YOUR_STUDENT_ID \
WANDB_NAME=TASK_3_1_1 \
lighteval vllm \
"model_name=Qwen/Qwen2.5-0.5B-Instruct,dtype=bfloat16,
max_model_length=4096,gpu_memory_utilization=0.9,
generation_parameters={max_new_tokens:4096,temperature:0.6,top_p:0.95}"
"lighteval|gsm8k|5"  --wandb --save-details
```

Please make sure to clone the latest version of the `lighteval` repository and **install it from scratch**, otherwise you may encounter unexpected bugs. Please also note that when running the `lighteval` framework, it is normal to encounter missing dependencies. Simply install the required packages manually if this occurs. (for example, **proper versions** of both `vllm` and `wandb` need to be installed manually). Please write down your score in the report.

(b) **Finetune the base model (5 point):** Use the `LLaMA-Factory` framework to finetune `Qwen2.5-0.5B-Instruct` on the `alpaca_en_demo` dataset. After correctly installing the `LLaMA-Factory` framework, you can finetune the model using the following script:

```
WANDB_API_KEY=YOUR_API_KEY \
```

```
WANDB_PROJECT=NLP2026_YOUR_STUDENT_ID \
WANDB_NAME=TASK_3_1_2 \
FORCE_TORCHRUN=1 llamafactory-cli train \
examples/train_full/llama3_full_sft.yaml
model_name_or_path=Qwen/Qwen2.5-0.5B-Instruct report_to=wandb
dataset=alpaca_en_demo  per_device_train_batch_size=32
```

Please make sure to clone the latest version of the `LLaMA-Factory` repository and **install it from scratch**, otherwise you may encounter unexpected bugs. Please also note that when running the `LLaMA-Factory` framework, it is normal to encounter missing dependencies. Simply install the required packages manually if this occurs. (for example, **proper versions** of both `deepspeed` and `wandb` need to be installed manually). You may need to adjust the hyperparameters according to the length of the training data and the available GPU memory. In your report, please also include the figure of the corresponding curve.

(c) **Choose your own training dataset (10 point):** It is not difficult to observe that the example dataset `alpaca_en_demo` is a simple and general-purpose dataset, which has little impact on improving the model's performance on mathematical reasoning datasets such as `GSM8K`. Learning to select appropriate datasets for model training is crucial.

In this part, we encourage you to select a public or private dataset of your choice to perform supervised fine-tuning (SFT) on the `Qwen2.5-0.5B-Instruct` model (you may also use LoRA fine-tuning). If you are unsure which dataset to choose, we recommend starting with a subset of `nvidia/OpenMathInstruct-2`. You may freely modify the training hyperparameters, although the default settings generally yield good results. Please keep the complete logs of all your experiments, and set `WANDB_NAME` to `TASK_3_1_3` for all runs. You may need to adjust the hyperparameters according to the length of the training data and the available GPU memory. Report the method you used to construct your dataset, and please also include the figure of the corresponding curve.

(d) **Evaluate your finetuned model (10 point):** In this section, you are required to evaluate the performance of your fine-tuned model on the `GSM8K` dataset. You may adjust the generation parameters as appropriate; it is not necessary to keep them identical to those used for the base model. When reporting to `wandb`, please set `WANDB_PROJECT` to `TASK_3_1_4`. You will find that improving the model's performance may require a large amount of data, and the Qwen series models are already well-trained. Therefore, as long as the fine-tuned model performs better than the base model, the result is considered acceptable. Write down the performance in the report.

Part 3-2: RL (10 points)

For 3-2, the instructions will not be as detailed as those for Task 3-1, but will instead provide a general outline.

Reproduce the basic characteristics of RL training (10 points): `simpleRL-reason` (https://github.com/hkust-nlp/simpleRL-reason) is a representative work built using the `verl` framework. This work uses the GRPO algorithm and is among the earliest studies to successfully reproduce the reinforcement learning training paradigm of DeepSeek-R1. Please use the `Qwen2.5-0.5B` model and any training dataset of your choice to reproduce the trends reported in `simpleRL-reason`, where the accuracy increases steadily as training progresses. As a default option, sampling a 1–3K-sized subset from `hkust-nlp/SimpleRL-Zoo-Data` is a good choice. When uploading your results to `wandb`, set `WANDB_NAME` to `TASK_3_2_1`. **In your report, please also include the figure of the corresponding curve.**

Part 4: Create a basic Unity Game (40 points)

Use the provided resources and any free online assets, do your best to build a game prototype with Unity. The game can be 3D or 2D, and the OS can be Windows or MacOS. For example, the game can be a basic FPS (like CS), or a basic 2D platformer (like MarioBro), or anything you like. We expect the work time to be around 3 or 4 hours.

Considering that this is the first Unity homework, we only expect a very basic game prototype, and **no novelty is expected or required**. And we do not expect it to have multiple levels. You will receive at least 30 points as long as your game is acceptable.

Submit the compiled game (which we can directly click and play) in `part4/game_winORmac`, along with a short 3-minute (no longer than 5 minutes) mp4 video recording of how this game can be played. You can add some explanation in the video but it's not expected or required.

In the report, you only need to briefly describe your game and add one or two screenshots.

Note: You can discuss with your team member but you should do this homework yourself.

```
part4/
|-- game_winORmac/
‘-- recording.mp4
```

Submission Requirement

Your submission should be organized in the following structure (the part3 folder should be empty):

```
submission_YOURSTUDENTID.zip
|-- report.pdf
|-- part1
|-- part2
`-- part3/4
```

Acknowledgement

Part 1 is inspired by CS224N: Natural Language Processing with Deep Learning (Stanford University) and was adapted from last year's assignment.

Part 2 is inspired by CS336: Language Models from Scratch (Stanford University).